

How to setup claude code in local system with local model and use that to create a python project



Setting up something like **Claude Code** locally with a **local model** is a bit tricky—because Claude itself is **not open-source and cannot run locally**. But you can absolutely recreate a *Claude-like coding assistant workflow* using local LLMs.

Here's the practical way to do what you want 🙌

1. Understand the Setup

You'll replace Claude with:

- A **local LLM** (runs on your machine)
- A **code assistant interface** (like Claude Code)

Popular stack:

- Local model runner → **Ollama**
- Code assistant UI → **Continue** or **Cursor**
- Models →
 - **Code Llama**
 - **DeepSeek Coder**
 - **Mistral**

2. Install Ollama (Local Model Runner)

On Linux / macOS:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Start service:

3. Pull a Coding Model

Good options:

```
ollama pull deepseek-coder
```

or

```
ollama pull codellama:7b
```

4. Test Locally

```
ollama run deepseek-coder
```

Try:

```
Write a Python script for a REST API using FastAPI
```



5. Install VS Code + Continue Extension

1. Install Visual Studio Code
2. Install Continue extension

6. Connect Continue to Ollama

Open Continue config (`~/.continue/config.json`) and add:

```
{  
  "models": [  

```

```
{
  "title": "Local DeepSeek",
  "provider": "ollama",
  "model": "deepseek-coder"
}
]
```

7. Create a Python Project (Claude-style workflow)

Now you can prompt like Claude Code:

Example prompts inside VS Code:

1. Initialize project

Create a Python project structure for a FastAPI app with:

- routes
- services
- models
- requirements.txt



2. Generate code

Create a FastAPI server with CRUD APIs for a Todo app



3. Improve code

Refactor this code to follow best practices and add type hints



Example Output Structure

Your model should generate:

```
project/
|— app/
|   |— main.py
|   |— routes/
|   |— models/
|   |— services/
|— requirements.txt
|— README.md
```



8. Run Your Python Project

Install dependencies:

```
pip install fastapi uvicorn
```

Run:

```
uvicorn app.main:app --reload
```

Optional: More “Claude-like” Experience

If you want a closer experience:

Try:

- [Cursor](#) → best Claude-like UX
- [Open WebUI](#) → chat interface
- [LangChain](#) → advanced workflows

Reality Check

- Local models are **weaker than Claude** (especially reasoning & long code)
- Need **good GPU** (8–16GB VRAM recommended)
- Expect slower responses

Summary

You cannot run Claude locally, but you can replicate it:

Claude Code → **Local Equivalent**

- Claude → DeepSeek / Code Llama
 - Claude Code → Continue / Cursor
 - API → Ollama
-

If you want, I can:

- Recommend the **best model for your hardware**
- Generate a **full Python project via prompts**
- Help you build a **fully offline AI coding agent**



Yes



Nice—let's tailor this properly so you get a **smooth, Claude-like local coding setup** instead of something slow or frustrating.

First, I need one key detail 👉

👉 **What are your system specs?**

- RAM (e.g., 8GB / 16GB / 32GB)
- GPU (if any, like NVIDIA RTX 3050, or none)
- CPU (optional but helpful)

🔧 **Meanwhile, here are 3 optimized setups (pick yours)**

🟢 **1. Low-end system (8GB RAM, no GPU)**

👉 Go lightweight or it will lag badly

Use:

- Ollama
- Model:
 - `deepseek-coder:1.3b` (*fast & usable*)
 - or `phi3` (*very lightweight*)

Command:

```
ollama pull deepseek-coder:1.3b
```

💡 **Expect:**

- Fast responses

- Basic coding help (not very smart)
-

2. Mid-range (16GB RAM, no/low GPU)

 Best balance (recommended for most people)

Use:

- DeepSeek Coder (6.7B or 7B)

```
ollama pull deepseek-coder:6.7b
```

 Expect:

- Good Python generation
 - Handles small projects well
 - Closest to Claude-like locally (without GPU)
-

3. High-end (GPU 8GB+ VRAM or 32GB RAM)

 This gets you close to Claude-level workflows

Use:

- deepseek-coder : 33b (if GPU strong)
- or mix with:
 - Mistral
 - Code Llama

 Expect:

- Much better reasoning
 - Can handle large Python projects
-

Best “Claude Code” Experience (IMPORTANT)

To really feel like Claude Code:

Option A (Best UX)

 Use Cursor

- Built-in AI coding assistant

- Works with Ollama (local models)
 - Feels very close to Claude
-

Option B (Free + Open Source)

👉 Use Continue + Visual Studio Code

⚡ Full Working Setup (Simple Path)

Step 1: Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
```

Step 2: Install model

```
ollama pull deepseek-coder:6.7b
```

Step 3: Install VS Code + Continue

Step 4: Configure model

```
{
  "models": [
    {
      "title": "Local Model",
      "provider": "ollama",
      "model": "deepseek-coder:6.7b"
    }
  ]
}
```



Ready-to-use Prompt (Copy This)

Paste inside Continue / Cursor:

- Modular folder structure
- PostgreSQL integration
- SQLAlchemy ORM
- Environment config
- Logging
- CRUD for Users



Also include:

- requirements.txt
- README.md



Want Next Level? (Optional)

If you want something *even closer to Claude Code agent*:

- Add `LangChain`
- Or `Open WebUI`
- Or build an **auto-coding agent** (like Devin-lite)



Honest Advice

- Don't go for huge models unless your system can handle it
- Start with **DeepSeek 6.7B** → best sweet spot
- Cursor > VS Code if you want Claude-like feel



Next step

Reply with your specs, and I'll:

- Pick the **exact best model**
- Give you a **perfect config**
- Generate a **complete Python project tailored to you**





Alright—straight up: with **8GB RAM + 2GB GPU**, you need to be *careful*. If you try bigger models, your system will crawl or crash.

But you can still build a **very usable local “Claude Code” setup** if you pick the right tools



Best Setup for YOUR System

Model (MOST IMPORTANT)

Use:

- DeepSeek Coder → 1.3b (*best choice*)
- OR
- Phi-3 → (*faster, lighter*)

Install:

```
ollama pull deepseek-coder:1.3b
```

 This will:

- Run smoothly on CPU
- Not depend on your small GPU
- Still generate decent Python code

Core Stack

- Runtime → Ollama
- Editor → Visual Studio Code
- AI plugin → Continue

 Avoid heavy tools like big models or multi-agent frameworks for now.

Continue Config (Optimized)

Use this minimal config:

```
{
  "models": [
    {
      "title": "Fast Local",
      "provider": "ollama",
      "model": "deepseek-coder:1.3b"
    }
  ],
  "completionOptions": {
    "temperature": 0.2,
    "maxTokens": 512
  }
}
```

💡 Keeps responses:

- Faster ⚡
- Less RAM usage
- More stable

How to Use It (Claude-style)

Because the model is small, **HOW** you prompt matters a lot.

❌ Bad prompt:

Build a full scalable backend system



✅ Good prompt:

Create a simple FastAPI project with:

- 1 route (GET /hello)
- Clean folder structure
- requirements.txt



👉 Then expand step-by-step:

Now add a SQLite database and a User model



Example Workflow (REALISTIC)

Step 1

Create a Python FastAPI project structure



Step 2

Add a Todo API with CRUD operations



Step 3

Refactor into services and models folders



👉 This “incremental prompting” is **critical** on small models.



Performance Tips (VERY IMPORTANT)

1. Close heavy apps

- Chrome tabs ❌
- Background apps ❌

2. Use smaller context

- Don't paste huge files
- Work file-by-file

3. Keep model warm

```
ollama run deepseek-coder:1.3b
```



Optional Upgrade (Huge Improvement)

If possible later:

- Upgrade RAM → 16GB
 - 👉 This alone will unlock:
 - deepseek-coder : 6.7b (MUCH smarter)
-



Honest Expectation

With your setup:

-  Good for small–medium Python projects
 -  Learning, prototyping
 -  Not great for large complex systems
 -  Not Claude-level reasoning
-

Want me to go further?

I can next:

- Build you a **complete Python project (step-by-step prompts)**
- Or create a **ready-to-run starter template**
- Or help you set up a **local coding agent (auto file editing)**

Just tell me 



How to give prompt? Do I need to create a .md ?



Good question—and this is where many people overcomplicate things.

 **Short answer:**

No, you **do NOT need to create a .md file** to give prompts.

But... using .md *can* help in some cases. Let me explain clearly 

1. The Normal Way (What You Should Do)

If you're using:

- Visual Studio Code
- Continue
- Ollama

 You just type prompts directly in the AI chat panel.

Example:

Create a simple Python FastAPI app with one route



That's it. No `.md` needed.

2. When `.md` Files ARE Useful

You use `.md` (Markdown) only if you want:

Persistent instructions (like Claude “system prompt”)

Create a file like:

project_rules.md



Example content:

You are a senior Python developer.

Rules:

- Use clean architecture
- Add type hints
- Keep code modular
- Use FastAPI

Then paste or reference it in chat:

Follow instructions from project_rules.md and create a project



Large structured prompts

For bigger instructions:

Requirements

- FastAPI
- SQLite
- CRUD

Structure

- routes/
- models/
- services/

Then paste into chat.

3. BEST METHOD (For Your Setup)

Since you're using a **small model (1.3B)**, do this instead:

- 👉 Don't rely on big `.md` files
 - 👉 Use **step-by-step prompts**
-

Ideal Workflow (IMPORTANT)

Step 1

Create a basic FastAPI project structure



Step 2

Add a `main.py` with one GET endpoint



Step 3

Add a Todo model using SQLite



Step 4

Create CRUD operations for Todo



- 👉 This works MUCH better than one big prompt.
-

4. Bonus: Inline Prompting (Powerful)

You can also highlight code in Visual Studio Code and ask:

Refactor this code



or

Add error handling



⚠️ Common Mistake

❌ Don't do this on your setup:

Build a complete production-grade scalable backend system with auth, DB, caching, logging



👉 Your model will:

- Get confused
- Give bad output

✅ Final Answer

- ❌ .md is NOT required
- ✅ Use chat directly
- ✅ Break prompts into small steps
- ✅ Use .md only for reusable instructions

If you want, I can give you:

👉 A **perfect prompt template** you can reuse

👉 Or simulate a **full Python project conversation** step-by-step like Claude Code

Just tell me 👍



perfect prompt template you can reuse



Got it—this is exactly what will make your setup feel much closer to Claude Code.

Since you're using a **small local model**, the trick is:

👉 **structured, consistent prompts** (not long messy ones)

Here's a **reusable prompt template** you can copy every time 👉



Universal Prompt Template (Best for Local Models)

Requirements:

- Keep code simple and clean
- Use type hints
- Follow good folder structure
- Add comments where needed



Output:

- Show file structure (if changed)
- Provide full code (only necessary files)
- Keep explanation short



Example 1 (Start Project)

- What I want now: initial structure



Requirements:

- Keep code simple
- Use main.py
- Add one GET endpoint

Output:

- Show folder structure
- Provide code



Example 2 (Add Feature)

- What already exists: basic app with one route
- What I want now: Todo CRUD



Requirements:

- Use SQLite
- Separate models and routes
- Keep it simple

Output:

- Only new/updated files



Example 3 (Refactor Code)

- Improve readability and structure



Requirements:

- Add type hints
- Split into functions
- Follow best practices

Output:

- Clean version of code
- Short explanation

Power Tips (VERY IMPORTANT for your system)

1. Always say “ONE task”

 Bad:

Build full backend with auth, DB, APIs



 Good:

Create user model



2. Use “What already exists”

 Prevents the model from rewriting everything

3. Limit output

Add:

Only show necessary files



4. Force structure

Add:

Show folder structure first



Optional Upgrade (Reusable System Prompt)

You can save this and reuse:

You are a strict, efficient Python backend engineer.



Rules:

- Do not overcomplicate
- Prefer simple solutions
- Keep responses short
- Only do exactly what is asked
- Avoid unnecessary features

How You'll Actually Use It

Inside Continue or Cursor:

1. Paste template
2. Fill blanks
3. Send
4. Repeat step-by-step

Pro Tip (Game Changer)

Instead of one big prompt, build like this:

1. Structure
2. One feature
3. Next feature
4. Refactor

 This is **exactly** how Claude Code works internally

If you want next level, I can give you:

- A **copy-paste prompt pack** (10+ real dev tasks)
- Or simulate a **full project build step-by-step** using this template

Just tell me 



Prompt to generate a code with python which detects a face and calculate the age and gender continuously with interval of 5s or while clicking esc button and show the ad based on the age and gender. It should detect single or multiple person. The output screen should be 2 screens left side should be camera which box the persons and show their age and gender and right side should be display ad.



Here's a **clean, reusable prompt** tailored for your setup (small local model + step-by-step generation). This will give you much better results than a single vague prompt.

Copy-Paste Prompt (Optimized for Your System)

- face detection
- age/gender prediction
- ad selection
- Use pre-trained models if possible (no training)



Output:

- Show folder structure
- Provide full working code
- Mention required libraries
- Keep explanation short

IMPORTANT (For Your System)

Your model may **struggle to generate everything perfectly in one go**.

👉 So break it like this:

Step-by-step prompts (BEST WAY)

Step 1

Create a Python script using OpenCV to detect faces from webcam and draw bounding boxes



Step 2

Add age and gender prediction using pre-trained models to the existing face detection code



Step 3

Modify the code to run detection every 5 seconds instead of every frame



Step 4

Split the screen into left (camera) and right (ad display)



Step 5

Add logic to display different ads based on age and gender



Pro Tip (Very Important)

Because you're using a small model:

👉 If output is messy, say:

Fix and simplify the previous code. Only show clean working version.



Libraries You'll Likely Need

Your model should use:

- OpenCV
 - NumPy
 - Pretrained models (like Caffe/TensorFlow age-gender models)
-

Reality Check

- Age/gender detection won't be 100% accurate
 - Real-time + 5s interval is good for performance
 - Your system can handle this if optimized
-

If you want, I can next:

- 👉 Generate the **FULL WORKING CODE** for you directly (optimized for your system)
- 👉 Or give you **ready-to-download models + setup steps**

Just tell me 👍



Generate the FULL WORKING CODE for you directly
(optimized for your system)



Alright—here's a **complete working prototype** tailored for your system (8GB RAM, low GPU).

It uses **lightweight models + 5s interval processing** so it won't freeze your machine.

What This Code Does

- Detects **single & multiple faces**
 - Predicts **age + gender**
 - Runs detection every **5 seconds**
 - Press **ESC** to exit
 - UI split:
 - **Left** → Camera + bounding boxes + labels
 - **Right** → Ad display (based on age/gender)
-

Required Libraries

Install first:

```
pip install opencv-python numpy
```

Folder Structure

```
| |— res10_300x300_ssd_iter_140000.caffemodel
| |— age_deploy.prototxt
| |— age_net.caffemodel
| |— gender_deploy.prototxt
| |— gender_net.caffemodel
| — ads/
| |— male_ad.jpg
| |— female_ad.jpg
| |— kids_ad.jpg
| |— senior_ad.jpg
```



👉 (I'll explain how to get models after code)

FULL WORKING CODE

```
)

gender_net = cv2.dnn.readNetFromCaffe(
    "models/gender_deploy.prototxt",
    "models/gender_net.caffemodel"
)

AGE_BUCKETS = ['(0-2)', '(4-6)', '(8-12)', '(15-20)',
               '(25-32)', '(38-43)', '(48-53)', '(60-100)']

GENDERS = ['Male', 'Female']
```

Where to Download Models

Search and download:

- res10_300x300_ssd_iter_140000.caffemodel
- deploy.prototxt
- age_net.caffemodel
- age_deploy.prototxt

- `gender_net.caffemodel`
- `gender_deploy.prototxt`

👉 These are standard OpenCV pretrained models (easy to find on GitHub).



Ads

Just add any images:

- `male_ad.jpg`
 - `female_ad.jpg`
 - `kids_ad.jpg`
 - `senior_ad.jpg`
-



Performance Tips (IMPORTANT for you)

- Keep camera at **640x480** ✓
 - Don't increase resolution ✗
 - 5s interval = good balance ✓
-



If you want next upgrade

I can help you:

- Replace with **more accurate models** (YOLO + Deep Learning)
- Add **tracking** (so ads don't flicker)
- Turn this into a **real product** (UI + deployment)

Just tell me 👍



Replace with more accurate models (YOLO + Deep Learning)
Add tracking (so ads don't flicker)
Turn this into a real product (UI + deployment)



Great—now you're moving from a prototype to something that actually feels like a **product**.
I'll upgrade your system in 3 layers:

1. Replace with Better Models (YOLO + Deep Learning)

Your current model is basic. Upgrade to:

- Face detection → YOLOv8
- Age/Gender → DeepFace

👉 Why:

- YOLO = fast + accurate
- DeepFace = better age/gender prediction than Caffe models

Install

```
pip install ultralytics deepface opencv-python numpy
```

Key Improvements

- Better detection in real-world lighting
- Handles multiple faces better
- More stable predictions

UPGRADED CODE (YOLO + DeepFace + Tracking)

```
try:
    analysis = DeepFace.analyze(face, actions=['age', 'gender'], enforce_detection=False)

    age = int(analysis[0]['age'])
    gender = analysis[0]['dominant_gender']

    face_memory[face_id] = (age, gender)

except:
    if face_id in face_memory:
```




2. Add Proper Tracking (No Flickering)

Current fix = basic memory



Better approach:

Use:

- Deep SORT

This gives:

- Persistent IDs per person
- Stable ads per person
- No flicker



If you want, I can integrate Deep SORT next (slightly heavier but better).



3. Turn Into Real Product (UI + Deployment)



Option A: Simple UI (Best for you)

Use:

- PyQt

You can create:

- Fullscreen display
 - Clean ad panel
 - Buttons (start/stop)
-



Option B: Web App (More scalable)

Use:

- FastAPI
- Frontend: HTML + JS



Camera → backend → results → UI



Option C: Production App

- Package with:
 - Docker
- Run on:
 - Raspberry Pi (light version)

- PC kiosk system

⚠️ Honest Reality (Important)

On your system:

- YOLOv8n ✅ works
- DeepFace ⚠️ slightly slow
- Tracking ✅ OK
- Full UI ⚠️ keep lightweight

🔥 Best Next Step

Don't jump everything at once. Do this:

1. Run YOLO + DeepFace version ✅
2. Then add tracking
3. Then UI

👉 I can take you further

Next I can give you:

- ✅ Deep SORT integrated version (no flicker at all)
- ✅ PyQt full UI app (looks like real product)
- ✅ Deployment guide (run as startup app / kiosk mode)

Just tell me what you want next 🙌



PyQt full UI app (looks like real product)



Perfect—this is where your project starts to look like a **real product** instead of a demo.

I'll give you a **clean PyQt app** with:

- Fullscreen UI

- Left: Camera + detection
 - Right: Ad display
 - Smooth updates (no flicker feel)
 - Start/Stop controls
-

Tech Stack

- UI → PyQt5
 - Vision → YOLOv8
 - AI → DeepFace
-

Install

```
pip install pyqt5 opencv-python ultralytics deepface numpy
```

Folder Structure

```
project/
|— app.py
|— ads/
|   |— male_ad.jpg
|   |— female_ad.jpg
|   |— kids_ad.jpg
|   |— senior_ad.jpg
```



FULL PYQT APP CODE

```
window = AIAdApp()
window.show()
sys.exit(app.exec_())
```



Make It Look Like a REAL PRODUCT



Fullscreen Mode

Add this line:

```
self.showFullScreen()
```



Hide Buttons (Kiosk Mode)

Remove buttons and auto-start camera.



Auto Start on Boot

Use:

- Linux → .desktop startup file
- Windows → Startup folder



Performance Tips (VERY IMPORTANT)

For your system:

- Keep YOLO → yolov8n.pt
- Keep interval → 5 sec
- Don't increase resolution



Next Level Upgrades

I can help you build:

- Deep SORT tracking (no flicker at all)
- Animated ads (video instead of images)

✓ Face recognition (repeat customer targeting 😎)

✓ Cloud dashboard (analytics + logs)

👉 What do you want next?

1. No flicker pro tracking (Deep SORT)
2. Modern UI (animations + transitions)
3. Turn into installable app (.exe / .deb)

Just tell me 👍

